Sistema de inventario y recomendación de productos para ferretería

Informe técnico | Prueba de innovación | 31 de octubre de 2026

Repositorio público: https://github.com/LucindaSerna/ferreteria-fullstack

1. Resumen ejecutivo

Se desarrolló un prototipo full-stack para consultar un catálogo de ferretería, gestionar compras e inventario y recomendar hasta tres productos complementarios. La solución combina una API REST en FastAPI, persistencia SQLite, una interfaz React y un modelo de asociaciones bayesianas calculado previamente a partir de tickets históricos.

El enfoque prioriza explicabilidad, funcionamiento con pocos datos y facilidad de reproducción. Las recomendaciones reflejan patrones de compra conjunta; no constituyen una verificación de compatibilidad técnica ni una inferencia causal.

2. Datos y alcance

• Catálogo de referencia: 28 productos, con SKU, descripción, categoría, precio y existencias.

• Historial de entrenamiento: 42 tickets de venta y 89 incidencias de producto en tickets.

• Espacio de asociaciones: 28 × 27 = 756 pares dirigidos de productos distintos.

• Artefactos versionados: Data/products.csv, Data/sales.csv y Data/asociaciones_modelo.csv. La base SQLite se genera localmente y no necesita incluirse en Git.

3. Arquitectura y flujo

• Frontend React/Vite: catálogo, carrito, sucursal y visualización de recomendaciones.

• Backend FastAPI: endpoints REST para productos, compras, recomendaciones y ajustes manuales de relaciones.

• SQLite: tablas de productos, compras, detalle de compras y relaciones manuales.

• Modelo offline: el notebook documenta el análisis y exporta las puntuaciones a CSV; la API consulta ese archivo y aplica filtros de disponibilidad y ajustes manuales.

Flujo principal: usuario selecciona producto → React consulta GET /recomendaciones/{sku} → FastAPI valida SKU, carga puntuaciones, descarta productos sin stock y ordena candidatos → interfaz presenta hasta tres sugerencias. En compras, el backend valida la solicitud y descuenta inventario dentro de una transacción.

4. Modelo matemático: asociación bayesiana

Para un producto de origen i y un producto candidato j, se estima la probabilidad de que j aparezca en un ticket dado que aparece i. Se usa un prior Beta para estabilizar las estimaciones cuando hay pocos tickets.

N = número total de tickets; n_i = tickets que contienen i; n_j = tickets que contienen j; n_ij = tickets que contienen ambos productos.

$$q_j = (n_j + \varepsilon) / (N + 2\varepsilon), \text{ con } \varepsilon = 0.5$$

$\theta_{ij} \sim \text{Beta}(k\,q_j,\ k(1-q_j))$, con $k = 5$

$E[\theta_{ij} \mid \text{datos}] = (n_{ij} + k\,q_j) / (n_i + k)$

La puntuación es la media posterior de una probabilidad condicional suavizada. El prior incorpora la popularidad general del producto candidato y evita extremos artificiales cuando n_i es pequeño. La dirección importa: la asociación $i \to j$ no necesariamente tiene el mismo valor que $j \to i$.

5. Selección de recomendaciones

• Se consideran candidatos distintos del producto de origen y presentes en el catálogo.

• Se excluyen productos sin existencias; la interfaz también evita sugerir artículos ya incluidos en el carrito.

• Las relaciones manuales guardadas en SQLite sustituyen la puntuación calculada para el par correspondiente.

• Se ordenan las puntuaciones de mayor a menor y se devuelven como máximo tres recomendaciones.

Interpretación: cuando existe una modificación manual, el valor es una puntuación de negocio y no debe describirse como probabilidad posterior estimada por el modelo.

6. Evaluación offline

Se realizó una evaluación leave-one-ticket-out (LOOTO) sobre las incidencias de producto en tickets, contrastando las sugerencias con los artículos restantes del ticket retenido. Los resultados reportados corresponden a 89 casos de evaluación, que no son observaciones independientes entre sí.

Métrica	Modelo bayesiano	Referencia por popularidad
Hit Rate@3	0.2697 (24/89)	0.0000
Precision@3	0.0936	0.0000
Recall@3	0.2397	0.0000

En la exploración de sensibilidad, $k = 1, 2, 5$ y 10 produjeron las mismas métricas reportadas; con $k = 20$ el desempeño disminuyó. Se eligió $k = 5$ como compromiso de suavización, sin atribuirle superioridad estadística demostrada.

Precaución: la muestra es pequeña y las incidencias de un mismo ticket están correlacionadas. Los resultados no permiten afirmar rendimiento futuro en producción ni una mejora estadísticamente significativa frente a otras alternativas.

7. Persistencia y controles operativos

• El archivo backend/init_db.py crea las tablas y carga el catálogo de referencia desde CSV.

• Las compras validan sucursal, SKU y cantidades positivas; el descuento de stock exige existencias suficientes.

• El registro de una compra utiliza una transacción SQLite para mantener consistencia entre cabecera, detalle y stock.

• La API ofrece operaciones CRUD de productos y administración de relaciones manuales; SQLite impone restricciones de integridad referencial.

8. Pruebas y reproducibilidad

Verificaciones realizadas: clonación de repositorio público; ejecución de init_db.py desde una copia limpia con Python 3.15; comprobación de 28 productos persistidos en SQLite. Durante el desarrollo también se verificaron respuestas de recomendaciones y un flujo de compra desde la interfaz.

Limitación de la prueba de instalación: la instalación limpia de paquetes no se pudo completar en el equipo utilizado porque Anaconda no cargó correctamente los módulos nativos _ssl y _sqlite3. Esta incidencia del entorno no constituye una validación completa de instalación desde cero. El repositorio incluye backend/requirements.txt para facilitarla en otro equipo.

Pasos para reproducir: clonar el repositorio; entrar a backend; crear y activar un entorno virtual con Python compatible; ejecutar python -m pip install -r requirements.txt; ejecutar python init_db.py; iniciar python -m uvicorn main:app --reload. En otra terminal, entrar a frontend, ejecutar npm install y npm run dev. La API se sirve normalmente en localhost:8000 y Vite en localhost:5173.

9. Limitaciones y oportunidades de mejora

• El volumen de tickets es reducido y puede producir asociaciones inestables o sesgadas hacia patrones particulares.

• La recomendación se basa en coocurrencia, no en propiedades físicas, compatibilidad entre piezas ni seguridad de uso.

• El modelo se calcula offline; cambios en ventas o catálogo requieren una actualización del artefacto de asociaciones.

• El prototipo no incorpora autenticación, roles, pruebas automatizadas extensas ni despliegue productivo.

• Próximas mejoras: ampliar datos, segmentar por sucursal y temporada, incorporar metadatos de compatibilidad, evaluar temporalmente y medir conversión real mediante pruebas A/B.

10. Decisiones de diseño y conclusión

Se eligió una arquitectura ligera (React + FastAPI + SQLite) por claridad, bajo costo operativo y facilidad de inspección. El enfoque Beta-Binomial permite explicar cada puntuación y regularizar estimaciones con pocos datos. La separación entre notebook, CSV de puntuaciones y servicio REST facilita reproducir el prototipo sin entrenar el modelo durante cada consulta.

El resultado es una prueba funcional y auditable del flujo de recomendación e inventario. Su principal valor es demostrar una solución de extremo a extremo con decisiones matemáticas explícitas, controles transaccionales y límites metodológicos reconocidos.